

Java Collection Framework - 12 Programs

Objective

To study and implement the major Collection Framework interfaces through separate simple Java programs, using commonly used methods and suitable implementation classes.

Programs Included

Collection, List, Set, SortedSet, NavigableSet, Queue, Deque, Map, SortedMap, NavigableMap, Iterator, and ListIterator.

System Requirements

Hardware: Computer/Laptop, minimum 4 GB RAM, keyboard and mouse.

Software: Java Development Kit (JDK), Java compiler, Java Runtime Environment (JRE), and any Java-supported IDE or terminal.

Note

Each program is kept independent and simple. Required classes are imported explicitly; wildcard imports are not used.

Program 1: Collection Interface

File Name

CollectionDemo.java

Aim / Objective

To demonstrate basic operations of the Collection interface using an ArrayList.

Algorithm

1. Create a Collection reference using ArrayList.
2. Add elements using add().
3. Display size and check an element using size() and contains().
4. Remove an element using remove().
5. Remove all elements using clear() and check isEmpty().

Source Code

```
import java.util.ArrayList;
import java.util.Collection;

public class CollectionDemo {
    public static void main(String[] args) {
        Collection<String> items = new ArrayList<>();

        items.add("Java");          // add() adds an element
        items.add("Python");
        items.add("C");

        System.out.println("Collection: " + items);
        System.out.println("Size: " + items.size()); // size() gives number of elements
        System.out.println("Contains Java: " + items.contains("Java")); // contains() checks an element

        items.remove("C");          // remove() removes an element
        System.out.println("After remove: " + items);

        items.clear();              // clear() removes all elements
        System.out.println("Is Empty: " + items.isEmpty()); // isEmpty() checks whether collection is empty
    }
}
```

Compilation and Execution

Compile: javac CollectionDemo.java

Execute: java CollectionDemo

Output Screen

```
TERMINAL

Collection: [Java, Python, C]
Size: 3
Contains Java: true
After remove: [Java, Python]
Is Empty: true
```

Result

The Collection Interface program was implemented successfully and its basic operations were demonstrated.

Program 2: List Interface

File Name

ListDemo.java

Aim / Objective

To demonstrate ordered List operations and duplicate elements.

Algorithm

1. Create a List using ArrayList.
2. Add elements and insert an element at an index.
3. Access an element using get().
4. Find the first position using indexOf().
5. Replace and remove elements using set() and remove().

Source Code

```
import java.util.ArrayList;
import java.util.List;

public class ListDemo {
    public static void main(String[] args) {
        List<String> names = new ArrayList<>();

        names.add("Rahul");           // add() adds an element
        names.add("Priya");
        names.add("Rahul");           // List allows duplicates
        names.add(1, "Arjun");        // add(index, value) inserts at a position

        System.out.println("List: " + names);
        System.out.println("Element at index 1: " + names.get(1)); // get() reads an element
        System.out.println("Index of Rahul: " + names.indexOf("Rahul")); // indexOf() finds first position

        names.set(0, "Kiran");        // set() replaces an element
        names.remove(2);               // remove(index) removes by position

        System.out.println("Updated List: " + names);
    }
}
```

Compilation and Execution

Compile: javac ListDemo.java

Execute: java ListDemo

Output Screen

```
TERMINAL

List: [Rahul, Arjun, Priya, Rahul]
Element at index 1: Arjun
Index of Rahul: 0
Updated List: [Kiran, Arjun, Rahul]
```

Result

The List Interface program was implemented successfully and its basic operations were demonstrated.

Program 3: Set Interface

File Name

SetDemo.java

Aim / Objective

To demonstrate Set operations and duplicate suppression.

Algorithm

1. Create a Set using HashSet.
2. Add several elements including a duplicate.
3. Display the Set.
4. Check an element and size.
5. Remove an element.

Source Code

```
import java.util.HashSet;
import java.util.Set;

public class SetDemo {
    public static void main(String[] args) {
        Set<Integer> numbers = new HashSet<>();

        numbers.add(10);           // add() adds an element
        numbers.add(20);
        numbers.add(30);
        numbers.add(20);           // duplicate is ignored

        System.out.println("Set: " + numbers);
        System.out.println("Contains 20: " + numbers.contains(20)); // contains() checks an element
        System.out.println("Size: " + numbers.size()); // size() gives number of elements

        numbers.remove(10);        // remove() removes an element
        System.out.println("After remove: " + numbers);
    }
}
```

Compilation and Execution

Compile: javac SetDemo.java

Execute: java SetDemo

Output Screen

```
TERMINAL

Set: [20, 10, 30]
Contains 20: true
Size: 3
After remove: [20, 30]
```

Result

The Set Interface program was implemented successfully and its basic operations were demonstrated.

Program 4: SortedSet Interface

File Name

SortedSetDemo.java

Aim / Objective

To demonstrate sorted set operations using TreeSet.

Algorithm

1. Create a SortedSet using TreeSet.
2. Add unsorted integer values.
3. Display the automatically sorted set.
4. Use first() and last().
5. Display headSet(), tailSet(), and subSet() ranges.

Source Code

```
import java.util.SortedSet;
import java.util.TreeSet;

public class SortedSetDemo {
    public static void main(String[] args) {
        SortedSet<Integer> numbers = new TreeSet<>();

        numbers.add(40);           // add() adds an element
        numbers.add(10);
        numbers.add(30);
        numbers.add(20);

        System.out.println("Sorted Set: " + numbers);
        System.out.println("First: " + numbers.first()); // first() gives smallest element
        System.out.println("Last: " + numbers.last());   // last() gives largest element

        System.out.println("Head Set: " + numbers.headSet(30)); // elements before 30
        System.out.println("Tail Set: " + numbers.tailSet(30)); // elements from 30
        System.out.println("Sub Set: " + numbers.subSet(10, 40)); // range of elements
    }
}
```

Compilation and Execution

Compile: javac SortedSetDemo.java

Execute: java SortedSetDemo

Output Screen

```
TERMINAL

Sorted Set: [10, 20, 30, 40]
First: 10
Last: 40
Head Set: [10, 20]
Tail Set: [30, 40]
Sub Set: [10, 20, 30]
```

Result

The SortedSet Interface program was implemented successfully and its basic operations were demonstrated.

Program 5: NavigableSet Interface

File Name

NavigableSetDemo.java

Aim / Objective

To demonstrate navigation operations on a sorted set.

Algorithm

1. Create a NavigableSet using TreeSet.
2. Add integer values.
3. Use higher() and lower() to find neighboring values.
4. Use ceiling() and floor() for boundary values.
5. Display the set in descending order.

Source Code

```
import java.util.NavigableSet;
import java.util.TreeSet;

public class NavigableSetDemo {
    public static void main(String[] args) {
        NavigableSet<Integer> numbers = new TreeSet<>();

        numbers.add(10);
        numbers.add(20);
        numbers.add(30);
        numbers.add(40);

        System.out.println("Navigable Set: " + numbers);

        System.out.println("Higher than 20: " + numbers.higher(20)); // next greater
        System.out.println("Lower than 20: " + numbers.lower(20));   // next smaller
        System.out.println("Ceiling of 25: " + numbers.ceiling(25)); // >= value
        System.out.println("Floor of 25: " + numbers.floor(25));     // <= value

        System.out.println("Descending Set: " + numbers.descendingSet()); // reverse order
    }
}
```

Compilation and Execution

Compile: javac NavigableSetDemo.java

Execute: java NavigableSetDemo

Output Screen

```
TERMINAL

Navigable Set: [10, 20, 30, 40]
Higher than 20: 30
Lower than 20: 10
Ceiling of 25: 30
Floor of 25: 20
Descending Set: [40, 30, 20, 10]
```

Result

The NavigableSet Interface program was implemented successfully and its basic operations were demonstrated.

Program 6: Queue Interface

File Name

QueueDemo.java

Aim / Objective

To demonstrate basic FIFO queue operations.

Algorithm

1. Create a Queue using LinkedList.
2. Add elements using offer().
3. View the front using peek().
4. Remove the front using poll().
5. Add another element and display the queue.

Source Code

```
import java.util.LinkedList;
import java.util.Queue;

public class QueueDemo {
    public static void main(String[] args) {
        Queue<String> queue = new LinkedList<>();

        queue.offer("A");           // offer() adds an element to the queue
        queue.offer("B");
        queue.offer("C");

        System.out.println("Queue: " + queue);
        System.out.println("Front: " + queue.peek()); // peek() reads the front element

        System.out.println("Removed: " + queue.poll()); // poll() removes the front element
        System.out.println("Queue after poll: " + queue);

        queue.add("D");             // add() also adds an element
        System.out.println("Final Queue: " + queue);
    }
}
```

Compilation and Execution

Compile: javac QueueDemo.java

Execute: java QueueDemo

Output Screen

```
TERMINAL

Queue: [A, B, C]
Front: A
Removed: A
Queue after poll: [B, C]
Final Queue: [B, C, D]
```

Result

The Queue Interface program was implemented successfully and its basic operations were demonstrated.

Program 7: Deque Interface

File Name

DequeDemo.java

Aim / Objective

To demonstrate insertion, viewing, and removal at both ends of a deque.

Algorithm

1. Create a Deque using ArrayDeque.
2. Add elements at both ends.
3. View the first and last elements.
4. Remove elements from both ends.
5. Display the remaining deque.

Source Code

```
import java.util.ArrayDeque;
import java.util.Deque;

public class DequeDemo {
    public static void main(String[] args) {
        Deque<String> deque = new ArrayDeque<>();

        deque.addFirst("B");           // addFirst() adds at the front
        deque.addLast("C");            // addLast() adds at the rear
        deque.addFirst("A");

        System.out.println("Deque: " + deque);
        System.out.println("First: " + deque.peekFirst()); // peekFirst() reads front
        System.out.println("Last: " + deque.peekLast());   // peekLast() reads rear

        deque.pollFirst();             // pollFirst() removes front
        deque.pollLast();              // pollLast() removes rear

        System.out.println("After removals: " + deque);
    }
}
```

Compilation and Execution

Compile: javac DequeDemo.java

Execute: java DequeDemo

Output Screen

TERMINAL

```
Deque: [A, B, C]
First: A
Last: C
After removals: [B]
```

Result

The Deque Interface program was implemented successfully and its basic operations were demonstrated.

Program 8: Map Interface

File Name

MapDemo.java

Aim / Objective

To demonstrate key-value operations using HashMap.

Algorithm

1. Create a Map using HashMap.
2. Add key-value pairs using put().
3. Retrieve a value using get().
4. Check keys and values.
5. Display keys, values, and entries.
6. Remove a mapping.

Source Code

```
import java.util.HashMap;
import java.util.Map;

public class MapDemo {
    public static void main(String[] args) {
        Map<Integer, String> students = new HashMap<>();

        students.put(101, "Rahul");          // put() adds a key-value pair
        students.put(102, "Priya");
        students.put(103, "Arjun");

        System.out.println("Map: " + students);
        System.out.println("Student 102: " + students.get(102)); // get() returns value
        System.out.println("Has key 101: " + students.containsKey(101)); // checks key
        System.out.println("Has value Rahul: " + students.containsValue("Rahul")); // checks value

        System.out.println("Keys: " + students.keySet());        // keySet() returns keys
        System.out.println("Values: " + students.values());       // values() returns values
        System.out.println("Entries: " + students.entrySet());    // entrySet() returns pairs

        students.remove(103);          // remove() removes a pair using its key
        System.out.println("After remove: " + students);
    }
}
```

Compilation and Execution

Compile: javac MapDemo.java

Execute: java MapDemo

Output Screen

TERMINAL

```
Map: {101=Rahul, 102=Priya, 103=Arjun}  
Student 102: Priya  
Has key 101: true  
Has value Rahul: true  
Keys: [101, 102, 103]  
Values: [Rahul, Priya, Arjun]  
Entries: [101=Rahul, 102=Priya, 103=Arjun]  
After remove: {101=Rahul, 102=Priya}
```

Result

The Map Interface program was implemented successfully and its basic operations were demonstrated.

Program 9: SortedMap Interface

File Name

SortedMapDemo.java

Aim / Objective

To demonstrate sorted key-value mappings using TreeMap.

Algorithm

1. Create a SortedMap using TreeMap.
2. Add key-value pairs.
3. Display the sorted map.
4. Use firstKey() and lastKey().
5. Display headMap(), tailMap(), and subMap() ranges.

Source Code

```
import java.util.SortedMap;
import java.util.TreeMap;

public class SortedMapDemo {
    public static void main(String[] args) {
        SortedMap<Integer, String> students = new TreeMap<>();

        students.put(103, "Arjun");      // put() adds a key-value pair
        students.put(101, "Rahul");
        students.put(102, "Priya");

        System.out.println("Sorted Map: " + students);
        System.out.println("First Key: " + students.firstKey()); // smallest key
        System.out.println("Last Key: " + students.lastKey());   // largest key

        System.out.println("Head Map: " + students.headMap(103)); // keys before 103
        System.out.println("Tail Map: " + students.tailMap(102)); // keys from 102
        System.out.println("Sub Map: " + students.subMap(101, 103)); // key range
    }
}
```

Compilation and Execution

Compile: javac SortedMapDemo.java

Execute: java SortedMapDemo

Output Screen

TERMINAL

```
Sorted Map: {101=Rahul, 102=Priya, 103=Arjun}
First Key: 101
Last Key: 103
Head Map: {101=Rahul, 102=Priya}
Tail Map: {102=Priya, 103=Arjun}
Sub Map: {101=Rahul, 102=Priya}
```

Result

The SortedMap Interface program was implemented successfully and its basic operations were demonstrated.

Program 10: NavigableMap Interface

File Name

NavigableMapDemo.java

Aim / Objective

To demonstrate navigation operations on sorted map keys.

Algorithm

1. Create a NavigableMap using TreeMap.
2. Add key-value pairs.
3. Use higherKey() and lowerKey().
4. Use ceilingKey() and floorKey().
5. Display the map in descending order.

Source Code

```
import java.util.NavigableMap;
import java.util.TreeMap;

public class NavigableMapDemo {
    public static void main(String[] args) {
        NavigableMap<Integer, String> students = new TreeMap<>();

        students.put(101, "Rahul");
        students.put(102, "Priya");
        students.put(103, "Arjun");
        students.put(104, "Kiran");

        System.out.println("Navigable Map: " + students);

        System.out.println("Higher Key than 102: " + students.higherKey(102)); // next greater
        System.out.println("Lower Key than 102: " + students.lowerKey(102)); // next smaller
        System.out.println("Ceiling Key of 102: " + students.ceilingKey(102)); // >= key
        System.out.println("Floor Key of 102: " + students.floorKey(102)); // <= key

        System.out.println("Descending Map: " + students.descendingMap()); // reverse order
    }
}
```

Compilation and Execution

Compile: javac NavigableMapDemo.java

Execute: java NavigableMapDemo

Output Screen

TERMINAL

```
Navigable Map: {101=Rahul, 102=Priya, 103=Arjun, 104=Kiran}
Higher Key than 102: 103
Lower Key than 102: 101
Ceiling Key of 102: 102
Floor Key of 102: 102
Descending Map: {104=Kiran, 103=Arjun, 102=Priya, 101=Rahul}
```

Result

The NavigableMap Interface program was implemented successfully and its basic operations were demonstrated.

Program 11: Iterator Interface

File Name

IteratorDemo.java

Aim / Objective

To demonstrate forward traversal of a collection using Iterator.

Algorithm

1. Create a List using ArrayList.
2. Add three names.
3. Create an Iterator using iterator().
4. Check the next element using hasNext().
5. Read each element using next().

Source Code

```
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

public class IteratorDemo {
    public static void main(String[] args) {
        List<String> names = new ArrayList<>();

        names.add("Rahul");
        names.add("Priya");
        names.add("Arjun");

        Iterator<String> it = names.iterator(); // iterator() creates an Iterator

        System.out.println("Elements using Iterator:");

        while (it.hasNext()) {                // hasNext() checks for next element
            String name = it.next();           // next() returns the next element
            System.out.println(name);
        }
    }
}
```

Compilation and Execution

Compile: javac IteratorDemo.java

Execute: java IteratorDemo

Output Screen

```
TERMINAL

Elements using Iterator:
Rahul
Priya
Arjun
```

Result

The Iterator Interface program was implemented successfully and its basic operations were demonstrated.

Program 12: ListIterator Interface

File Name

ListIteratorDemo.java

Aim / Objective

To demonstrate forward and backward traversal of a List using ListIterator.

Algorithm

1. Create a List using ArrayList.
2. Add three names.
3. Create a ListIterator using listIterator().
4. Traverse forward using hasNext() and next().
5. Traverse backward using hasPrevious() and previous().

Source Code

```
import java.util.ArrayList;
import java.util.List;
import java.util.ListIterator;

public class ListIteratorDemo {
    public static void main(String[] args) {
        List<String> names = new ArrayList<>();

        names.add("Rahul");
        names.add("Priya");
        names.add("Arjun");

        ListIterator<String> it = names.listIterator(); // creates ListIterator

        System.out.println("Forward Direction:");

        while (it.hasNext()) {
            System.out.println(it.next()); // hasNext() checks forward element
            // next() moves forward
        }

        System.out.println("Backward Direction:");

        while (it.hasPrevious()) {
            System.out.println(it.previous()); // hasPrevious() checks previous element
            // previous() moves backward
        }
    }
}
```

Compilation and Execution

Compile: javac ListIteratorDemo.java

Execute: java ListIteratorDemo

Output Screen

TERMINAL

```
Forward Direction:  
Rahul  
Priya  
Arjun  
Backward Direction:  
Arjun  
Priya  
Rahul
```

Result

The ListIterator Interface program was implemented successfully and its basic operations were demonstrated.